Informe Técnico: Taller de Programación Declarativa

Estudiante: Raul A. Alzate **Curso:** Modelos y Paradigmas de Programación **Fecha de Entrega:** 23/02/2026

---

1. Funciones vs. Procedimientos: Por qué el `if` sin `else` lanza excepción en funciones y no en procedimientos

En Oz, una función es **azúcar sintáctico** sobre un procedimiento con un argumento de salida implícito `?Result`. La declaración:

```
None
fun {F X}
  if Cond then ValorA
  end
end
```

se expande internamente a:

```
None
proc {F X ?Result}
  if Cond then Result = ValorA
  end
end
```

Perspectiva funcional: Si `Cond` es falsa y no hay `else`, la variable `Result` del Almacén de Asignación Única (SAS) nunca recibe un valor. El sistema intenta unificarla con el valor de retorno que la expresión llamadora espera, pero esa variable está sin ligar. Dado que Oz usa variables de flujo de datos (*dataflow variables*), la ejecución quedaría **suspendida indefinidamente**, lo cual es un error semántico. Mozart lo reporta lanzando una excepción. Este comportamiento es correcto porque una función matemática debe producir un resultado para cada entrada válida; si no puede producirlo, es una violación del contrato y merece una excepción.

Perspectiva procedimental: Los procedimientos no tienen variable de retorno implícita. Un `if` sin `else` en un procedimiento simplemente ejecuta la instrucción vacía `skip` cuando la condición es falsa. No hay ninguna variable que deba ser ligada, así que no hay violación semántica.

---

2. `max.oz` — Conversión a notación procedimental

El enunciado pide sustituir cada `fun` por `proc` con un parámetro de salida explícito `?`.

```
None
proc {Max L ?Result}
   proc {MaxAcc Remaining CurrentMax ?Res}
      case Remaining
      of nil then Res = CurrentMax
      [] H|T then
         if H > CurrentMax then {MaxAcc T H Res}
         else {MaxAcc T CurrentMax Res}
         end
      end
   end
in
   case L
   of nil then raise emptyList end
   [] H|T then {MaxAcc T H Result}
   end
end
```

Justificación de la traducción:

- `fun {Max L}` → `proc {Max L ?Result}`: el valor de retorno se convierte en el parámetro `?Result`.
- `fun {MaxAcc L M}` → `proc {MaxAcc Remaining CurrentMax ?Res}`: mismo principio.
- Toda aplicación `{MaxLoop T H}` que antes retornaba un valor ahora recibe la variable de salida: `{MaxAcc T H Res}`.
- La comparación `if M > H` controla qué valor se pasa como nuevo `CurrentMax`; cuando la lista se agota (`nil`), se unifica `Res = CurrentMax`.

---

3. `factList.oz` — Lista incremental de factoriales

`{FactList N}` recibe un **entero** N y retorna `[1! 2! ... N!]`. La computación es recursiva (la llamada recursiva está en la cola del acumulador).

```
None
fun {Reverse L}
   fun {RevAcc L Acc}
      case L of nil then Acc
      [] H|T then {RevAcc T H|Acc}
      end
   end
in {RevAcc L nil} end

fun {FactList N}
   fun {FactLoop I CurrFact Acc}
      if I > N then {Reverse Acc}
      else
         local NextFact = CurrFact * I in
            {FactLoop I+1 NextFact NextFact|Acc}
         end
      end
   end
in
   if N < 1 then raise illegalArgument end
   else {FactLoop 1 1 nil}
   end
end
```

Traza: `{FactList 4}`:

I	CurrFact	Acc (antes de Reverse)
1	1	[1]
2	2	[2, 1]
3	6	[6, 2, 1]
4	24	[24, 6, 2, 1]
5 > 4	—	`{Reverse [24,6,2,1]}` = **[1 2 6 24]** ✓

El acumulador crece por la cabeza (O(1) por inserción) y `Reverse` lo corrige al final en O(n).

---

4. `sum.oz` — Sumatoria iterativa

Por qué la implementación original no es iterativa

```
None
fun {Sum N}
  if N == 0 then 0
  else N + {Sum N-1}  % <- operacion pendiente en la pila
  end
end
```

La expresión `N + {Sum N-1}` deja la adición **suspendida en la pila semántica** hasta recibir el resultado recursivo. Esto genera un crecimiento lineal O(N) en la pila. **No es recursión de cola.**

Versión iterativa con iterador de alto orden

Para alcanzar Nivel 3 en el criterio de soluciones iterativas, la implementación usa el iterador de alto orden `{FoldRange}` definido como una abstracción reutilizable:

```
None
% Iterador de alto orden: reduce el rango [Low..High] con F y acumulador Acc
fun {FoldRange Low High F Acc}
   if Low > High then Acc
   else {FoldRange Low+1 High F {F Acc Low}}
   end
end

fun {Sum N}
   if N < 0 then raise illegalArgument end
   else {FoldRange 1 N fun {$ Acc X} Acc+X end 0}
   end
end
```

`{FoldRange}` es genérico: recibe cualquier función `F` de dos argumentos y opera sobre cualquier rango entero. `{Sum}` lo instancia con la función anónima `fun {$ Acc X} Acc+X end`.

Modelo formal (sistema de transición de estados)

El estado es la tupla **S = (I, Acc)** donde I es el índice actual del rango:

Elemento	Valor
S_0	(1, 0) — I empieza en 1, acumulador vacío
$S_i \rightarrow S_{i+1}$	(I, Acc) → (I+1, Acc + I)
S_final	(N+1, N·(N+1)/2)
Invariante P(I, Acc)	`Acc = (I-1)·I/2` (Acc contiene la suma de 1 hasta I−1)
Progreso	I crece estrictamente → se alcanza I=N+1 en exactamente N pasos

Verificación del invariante:

- En S_0: `Acc=0 = (1-1)·1/2 = 0` ✓
- Si P(I, Acc) vale, en S_{i+1}=(I+1, Acc+I): `Acc+I = (I-1)·I/2 + I = I·(I+1)/2 = ((I+1)-1)·(I+1)/2` ✓
- En S_final (I=N+1): `Acc = N·(N+1)/2 = Sum(N)` ✓

Es iterativa porque `{FoldRange Low+1 High F {F Acc Low}}` es la **última** instrucción evaluada; no hay operaciones pendientes → Mozart aplica TCO → pila semántica O(1).

---

5. `alltail.oz` — ForAllTail

```
None
proc {ForAllTail Xs P}
   case Xs of nil then skip
   [] _|T then
      {P Xs}
      {ForAllTail T P}
   end
end
```

Correctitud: El `case` garantiza que `P` solo se aplica cuando `Xs` no es `nil`. En cada paso se aplica `P` a la sublista actual y se continúa con la cola `T`, que es estrictamente más corta. Para `Xs = [X1...Xn]` se producen exactamente las aplicaciones:

```
None
{P [X1 X2 ... Xn]}
{P [X2 ... Xn]}
...
{P [Xn]}
```

Complejidad espacial: La llamada recursiva `{ForAllTail T P}` es la última instrucción del bloque `_|T`. Si `P` no deja trabajo pendiente, es recursión de cola → O(1) en pila.

---

6. `convolucion.oz` — Convolución simbólica con recursión de cola

```
None
fun {Reverse L}
   fun {RevAcc L Acc}
      case L of nil then Acc
      [] H|T then {RevAcc T H|Acc}
      end
   end
in {RevAcc L nil} end

fun {Convolucion L1 L2}
   fun {ConvAcc L1 RevL2 Acc}
      case L1#RevL2
      of nil#_ then {Reverse Acc}
      [] (H1|T1)#(H2|T2) then
         {ConvAcc T1 T2 (H1#H2)|Acc}
      end
   end
in
   {ConvAcc L1 {Reverse L2} nil}
end
```

El operador `#`: Crea la tupla simbólica `'#'(H1 H2)`. El resultado `[x1#yn  x2#yn-1 ...  xn#y1]` es una lista de pares simbólicos, no valores numéricos.

Por qué es recursión de cola y realiza exactamente N invocaciones: `{ConvAcc T1 T2 ...}` es la única llamada recursiva y es la **última instrucción evaluada**. Los parámetros `T1` y `T2` tienen exactamente un elemento menos en cada paso. Partiendo de listas de longitud N, se realizan exactamente **N llamadas** recursivas antes de alcanzar `nil`.

Traza para `{Convolucion [x1 x2 x3] [y1 y2 y3]}`:

Paso	L1	RevL2	Acc (en construcción)
1	[x1,x2,x3]	[y3,y2,y1]	[x1#y3]
2	[x2,x3]	[y2,y1]	[x2#y2, x1#y3]
3	[x3]	[y1]	[x3#y1, x2#y2, x1#y3]
4	nil	—	`Reverse` → **[x1#y3 x2#y2 x3#y1]** ✓

---

7. `fibo.oz` — Fibonacci recursivo e iterativo

Versión recursiva intuitiva

```
None
fun {FiboRec N}
   if N < 2 then 1
   else {FiboRec N-1} + {FiboRec N-2}
   end
end
```

Por qué es recursiva (no iterativa): `{FiboRec N-1} + {FiboRec N-2}` genera **dos** llamadas recursivas y la adición queda suspendida. Esto produce un árbol de llamadas de tamaño $O(2^N)$ → pila $O(2^N)$. En este computador, la versión recursiva corre en tiempo razonable hasta aproximadamente **N = 35**.

Versión iterativa

```
None
fun {Fibo N}
   fun {FiboAcc K Curr Prev}
      if K == N then Curr
      else {FiboAcc K+1 Curr+Prev Curr}
      end
   end
in
   if N < 2 then 1
```

```
   else {FiboAcc 1 1 1}
   end
end
```

Modelo formal

El estado es **S = (K, Curr, Prev)**:

Elemento	Valor
S_0	(1, 1, 1) = (1, fib(1), fib(0))
$S_i \rightarrow S_{i+1}$	(K, Curr, Prev) → (K+1, Curr+Prev, Curr)
S_final	(N, fib(N), fib(N-1))
Invariante P(K, Curr, Prev)	`Curr = fib(K)  ∧  Prev = fib(K-1)`
Progreso	K crece de 1 a N → se alcanza el estado final en N−1 pasos

Verificación del invariante: En cada paso, el nuevo `Curr = Curr + Prev = fib(K) + fib(K-1) = fib(K+1)`, y el nuevo `Prev = fib(K)`. El invariante se preserva. En S_final, `Curr = fib(N)` es el resultado correcto.

Pruebas con N = 35:

```
None
{Browse {Fibo 36}}    % Fib(N+1)
{Browse {Fibo 350}}   % Fib(10*N)
{Browse {Fibo 3500}}  % Fib(100*N)
```

La versión iterativa los calcula instantáneamente gracias a TCO: pila O(1), tiempo O(N).

---

8. `add.oz` — Suma de números binarios

```
None
fun {Add B1 B2}
```

```
   fun {AddAcc R1 R2 Carry Acc}
      case R1#R2
      of nil#nil then
         if Carry == 0 then Acc
         else Carry|Acc end
      [] (H1|T1)#(H2|T2) then
         local Sum = H1 + H2 + Carry in
            {AddAcc T1 T2 Sum div 2 (Sum mod 2)|Acc}
         end
      end
   end
in
   {AddAcc {Reverse B1} {Reverse B2} 0 nil}
end
```

Diseño: La suma binaria requiere propagar el *carry* del bit menos significativo (LSB) al más significativo (MSB). Las listas tienen MSB primero, así que se invierten antes de procesar. Al construir el resultado con `|` (prepend) durante el recorrido LSB→MSB, el acumulador queda automáticamente en orden MSB-primero al finalizar.

Traza para `{Add [1 1 0 1 1 0] [0 1 0 1 1 1]}` (54 + 23 = 77):

Bit pos.	H1	H2	Carry in	Sum	Bit	Carry out
0 (LSB)	0	1	0	1	1	0
1	1	1	0	2	0	1
2	1	1	1	3	1	1
3	0	0	1	1	1	0
4	1	1	0	2	0	1
5 (MSB)	1	0	1	2	0	1
carry	—	—	1	—	1	—

Resultado: **[1 0 0 1 1 0 1]** = 1001101_2 = 77 ✓

---

9. `newton.oz` — Método de Newton Genérico

a) `{Derivar F A}` — Derivación simbólica

Implementa las 8 reglas del Cuadro 1 mediante *pattern matching* sobre la etiqueta de la fórmula:

```
None
fun {Derivar F A}
   if {IsFloat F} then 0.0
   elseif {IsAtom F} then
      if F == A then 1.0 else 0.0 end
   else
      case {Label F}
      of s then s({Derivar F.1 A} {Derivar F.2 A})          % (f+g)'=f'+g'
      [] r then r({Derivar F.1 A} {Derivar F.2 A})          % (f-g)'=f'-g'
      [] m then s(m({Derivar F.1 A} F.2)                    % (f*g)'=f'g+fg'
                  m(F.1 {Derivar F.2 A}))
      [] d then d(r(m({Derivar F.1 A} F.2)                  %
(f/g)'=(f'g-fg')/g²
                    m(F.1 {Derivar F.2 A}))
                 e(F.2 2.0))
      [] l then d({Derivar F.1 A} F.1)                      % ln(f)'=f'/f
      [] e then m(e(F.1 F.2)                                %
(f^g)'=f^g*(f'g/f+g'ln f)
                  s(d(m({Derivar F.1 A} F.2) F.1)
                    m({Derivar F.2 A} l(F.1))))
      end
   end
end
```

Verificación: `{Derivar s(k m(3.0 x)) x}` aplica regla (3) → `s(Dk, Dm(3.0 x))` donde `Dk=0.0` (k es átomo constante) y `Dm(3.0 x)` aplica regla (5) → `s(m(0.0 x) m(3.0 1.0))`. Resultado: `s(0.0 s(m(0.0 x) m(3.0 1.0)))` ✓

b) `{Evaluar F A V}` — Evaluación numérica

Sustituye cada ocurrencia del átomo `A` por el flotante `V` y evalúa la expresión aritméticamente recorriendo el árbol de la fórmula:

```
None
fun {Evaluar F A V}
   if {IsFloat F} then F
   elseif {IsAtom F} then
      if F == A then V else raise undefinedVar(F) end
```

```
   else
      case {Label F}
      of s then {Evaluar F.1 A V} + {Evaluar F.2 A V}
      [] r then {Evaluar F.1 A V} - {Evaluar F.2 A V}
      [] m then {Evaluar F.1 A V} * {Evaluar F.2 A V}
      [] d then {Evaluar F.1 A V} / {Evaluar F.2 A V}
      [] e then {Float.pow {Evaluar F.1 A V} {Evaluar F.2 A V}}
      [] l then {Float.log {Evaluar F.1 A V}}
      end
   end
end
```

Verificación: `{Evaluar d(2.0 x) x 4.0}` → `2.0 / 4.0 = 0.5` ✓

c) `{Limpiar F}` — Simplificación algebraica

Elimina neutrales y absorbentes: `f+0=f`, `0+f=f`, `f-0=f`, `0*f=0`, `f*0=0`, `1*f=f`, `f*1=f`, `0/g=0`.

```
None
fun {Limpiar F}
   if {IsFloat F} orelse {IsAtom F} then F
   else
      case {Label F}
      of s then
         local F1={Limpiar F.1} F2={Limpiar F.2} in
            if F1==0.0 then F2 elseif F2==0.0 then F1 else s(F1 F2) end
         end
      [] r then
         local F1={Limpiar F.1} F2={Limpiar F.2} in
            if F2==0.0 then F1 else r(F1 F2) end
         end
      [] m then
         local F1={Limpiar F.1} F2={Limpiar F.2} in
            if     F1==0.0 orelse F2==0.0 then 0.0
            elseif F1==1.0 then F2
            elseif F2==1.0 then F1
            else m(F1 F2) end
         end
      [] d then
         local F1={Limpiar F.1} F2={Limpiar F.2} in
            if F1==0.0 then 0.0 else d(F1 F2) end
         end
      [] e then e({Limpiar F.1} {Limpiar F.2})
      [] l then l({Limpiar F.1})
```

```
      end
   end
end
```

Verificación: `{Limpiar s(0.0 s(m(0.0 x) m(3.0 1.0)))}`:

- `s(m(0.0 x) m(3.0 1.0))`: el término `m(0.0 x)` limpia a `0.0`, entonces `s(0.0 m(3.0 1.0))` → `m(3.0 1.0)`.
- `s(0.0 m(3.0 1.0))` → `m(3.0 1.0)`. Además `m(3.0 1.0)` → `3.0` ✓

d) `{Raiz F X0 BuenaAprox}` — Método de Newton

```
None
fun {Raiz F X0 BuenaAprox}
   local A  = x
         DF = {Derivar F A}
   in
      fun {NewtonLoop Xn}
         if {BuenaAprox F Xn} then Xn
         else
            local FXn  = {Evaluar F  A Xn}
                  DFXn = {Evaluar DF A Xn}
            in
               {NewtonLoop Xn - FXn/DFXn}
            end
         end
      end
   in
      {NewtonLoop X0}
   end
end
```

La iteración de Newton es: **X□₊₁ = X□ − F(X□)/F'(X□)**. La derivada se calcula **simbólicamente una sola vez** con `Derivar` antes de entrar al ciclo y se evalúa numéricamente en cada iteración con `Evaluar`. `BuenaAprox` es una función de primer orden inyectada desde el exterior — esto logra un **desacoplamiento total** entre el motor numérico y el criterio de parada.

Ejemplo: Raíz de f(x) = x² − 2.0 con X_0=1.0 → converge a √2 ≈ 1.41421.

e) `FormulaOps` — Registro que alberga procedimientos

Para demostrar el concepto de **embebimiento**, las cuatro operaciones se encapsulan en un único registro de primer orden:

```
None
FormulaOps = ops(
   derivar: fun {$ F A}   {Derivar F A}   end
   evaluar: fun {$ F A V} {Evaluar F A V} end
   limpiar: fun {$ F}     {Limpiar F}     end
   raiz:    fun {$ F X0 BuenaAprox} {Raiz F X0 BuenaAprox} end
)
```

`FormulaOps` es un registro cuyos **campos son funciones**. Puede pasarse como argumento, almacenarse en otra estructura o ligarse a una variable, exactamente igual que cualquier valor de Oz.

Uso:

```
None
local Deriv = FormulaOps.derivar
      Eval  = FormulaOps.evaluar
in
   {Browse {Deriv s(k m(3.0 x)) x}}     % derivada simbolica
   {Browse {Eval  d(2.0 x) x 4.0}}      % evaluacion numerica -> 0.5
end
```

Por qué cumple el criterio de embebimiento: el registro agrupa comportamiento (procedimientos) junto con su nombre semántico en una sola entidad. Esto desacopla el "qué hacer" (el registro) del "cómo invocarlo" (el motor de Newton), permitiendo sustituir cualquier operación sin modificar `Raiz`.

---

10. Análisis de Complejidad Espacial: Recursión vs. Iteración

Optimización de Llamada de Cola (TCO)

En `{ForAllTail}`, `{Sum}`, `{Fibo}`, `{Add}` y `{Convolucion}`, la llamada recursiva es la **última instrucción evaluada**: no hay operaciones pendientes fuera de ella. Mozart aplica TCO reutilizando el mismo marco de entorno. La pila semántica mantiene tamaño **O(1)** sin importar el tamaño de la entrada.

Recursión clásica (sin TCO)

En `{FiboRec N}`, la expresión `{FiboRec N-1} + {FiboRec N-2}` genera dos llamadas pendientes. El tamaño de la pila crece como $\mathbf{O(2^N)}$ — impracticable para N > 35. Este contraste motiva la versión iterativa de Fibonacci.

---

11. Traducción al Lenguaje Núcleo

Las funciones de Oz son azúcar sintáctico sobre las 8 sentencias del kernel language. A continuación la expansión estricta de `{Sum}`:

```
None
local Sum in
   Sum = proc {$ L ?Result}
      local SumAcc in
         SumAcc = proc {$ K Acc ?AccResult}
            local Cond in
               Cond = (K == 0)
               if Cond then AccResult = Acc
               else
                  local NewAcc in
                     NewAcc = Acc + K
                     local K1 in
                        K1 = K - 1
                        {SumAcc K1 NewAcc AccResult}
                     end
                  end
               end
            end
         end
         {SumAcc L 0 Result}
      end
   end
end
```

Análisis:

1. Las funciones son `proc {$ ... ?Salida}` — procedimientos anónimos.
2. Cada operación intermedia requiere una `local ... in` explícita — ilustra el Almacén de Asignación Única.
3. El `if` del kernel language exige que la condición sea una variable ligada a un booleano — de ahí `Cond = (K == 0)`.
4. No hay `fun`, no hay expresiones compuestas: toda la semántica se reduce a unificación y aplicación de procedimientos.